
EXAMEN PARCIAL

PROGRAMACIÓN ORIENTADA A OBJETOS II

Docente: Mg. Aldo H. Zanabria Gálvez

Duración: 90 minutos

Indicaciones: Responda con claridad. Justifique cuando se solicite. En las preguntas de programación, utilice código limpio, ordenado y correctamente estructurado.

SECCIÓN I: CONCEPTOS TEÓRICOS

1. Defina el concepto de polimorfismo y mencione sus tipos.

Concepto:

polimorfismo permite que una misma llamada a una función ejecute comportamientos diferentes según el tipo de objeto que la invoque. C++ divide este concepto en dos categorías principales: estático (en tiempo de compilación) y dinámico (en tiempo de ejecución).

Tipos:

A. Polimorfismo Dinámico (Tiempo de ejecución)

Se logra mediante la herencia y el uso de funciones virtuales (virtual). Permite tratar a objetos de clases derivadas como si fueran de la clase base, ejecutando el método correcto según el objeto real.

Requisito: Requiere el uso de punteros (*) o referencias (&) a la clase base.

Palabra clave virtual: Indica al compilador que la resolución del método debe hacerse en tiempo de ejecución (usando una tabla virtual o VTable).

Palabra clave override: Asegura explícitamente que estás sobrescribiendo un método de la clase base.

B. Polimorfismo Estático (Tiempo de compilación).

No requiere herencia ni funciones virtuales. El compilador decide qué función llamar antes de que el programa se ejecute, lo que lo hace más rápido porque no tiene penalización de rendimiento en ejecución.

Sobrecarga de funciones: Funciones con el mismo nombre, pero diferentes parámetros.

Sobrecarga de operadores: Definir cómo actúan los símbolos (+, -, <<) con clases personalizadas.

Plantillas (Templates): Código genérico que se adapta a cualquier tipo de datos.

2. Diferencie entre herencia simple y herencia múltiple.

La herencia en C++ es un pilar de la Programación Orientada a Objetos (POO) que permite a una clase nueva (clase derivada o hija) adquirir los atributos y métodos de una clase existente (clase base o padre). Su objetivo principal es la reutilización de código y la creación de relaciones jerárquicas lógicas entre estructuras.

Tipos de herencia

- **Herencia simple:** Una clase hija hereda de una única clase padre
- **Herencia múltiple:** Una clase hija puede heredar directamente de dos o mas clases base

3. ¿Qué es una clase abstracta y para qué se utiliza?

Una **clase abstracta** en C++ es una clase diseñada exclusivamente para servir como base de otras clases, lo que significa que no se pueden crear objetos (instancias) directamente de ella. Su propósito principal es definir una interfaz común, un "molde" o un contrato que todas sus clases derivadas deben cumplir de forma obligatoria.

¿Cómo se define?

Una clase se convierte en abstracta de forma automática cuando contiene al menos una función virtual pura. Una función virtual pura es un método que se declara en la clase base, pero no tiene implementación; en su lugar, se iguala a cero (= 0).

¿Para qué se utiliza?

- Definir contratos obligatorios: Fuerza a todas las clases hijas a implementar el método virtual puro. Si una clase hija no redefine dicho método, esa clase hija también se vuelve abstracta y no se podrá usar.
- Habilitar el polimorfismo: Permite crear colecciones (arreglos o vectores) de punteros de la clase base abstracta que apunten a objetos de distintas clases hijas, ejecutando el comportamiento correcto en tiempo de ejecución.
- Abstracción de conceptos: Permite modelar conceptos teóricos que no existen por sí mismos en el mundo real (por ejemplo, no puedes tocar una "Forma", pero sí puedes tocar un "Círculo" o un "Cuadrado").

4. Explique el concepto de encapsulamiento con un ejemplo.

El encapsulamiento es un principio fundamental de la Programación Orientada a Objetos (POO) que consiste en ocultar los detalles internos de un objeto y proteger sus datos de modificaciones no autorizadas.

¿Cómo funciona?

- **Datos privados (private):** Las variables de la clase se ocultan del exterior.
- **Métodos públicos (public):** Se usan funciones especiales para acceder y modificar los datos de forma segura.
- **Control:** Permite validar los datos antes de guardarlos.

Ejemplo práctico en C++

Imaginemos una cuenta bancaria. No podemos permitir que cualquiera cambie el saldo directamente a un número negativo. Debemos obligar a usar métodos que validen la operación.

```

#include <iostream>
#include <string>

class CuentaBancaria {
private:
    // Datos ocultos y protegidos
    std::string titular;
    double saldo;

public:
    // Constructor para inicializar el objeto
    CuentaBancaria(std::string nombre, double saldoInicial) {
        titular = nombre;
        if (saldoInicial >= 0) {
            saldo = saldoInicial;
        } else {
            saldo = 0; // Validación básica
        }
    }

    // Método "Getter" para leer el saldo de forma segura
    double getSaldo() const {
        return saldo;
    }

    // Método para modificar el saldo bajo reglas estrictas
    void depositar(double cantidad) {
        if (cantidad > 0) {
            saldo += cantidad;
            std::cout << "Deposito exitoso de: $" << cantidad << "\n";
        } else {
            std::cout << "Error: La cantidad debe ser positiva.\n";
        }
    }
};

int main() {
    // Creación del objeto
    CuentaBancaria miCuenta("Juan Perez", 500.0);

    // INTENTO INCORRECTO: miCuenta.saldo = -1000;
    // Compilación fallida: 'saldo' es privado.

    // FORMA CORRECTA: Uso de la interfaz pública
    miCuenta.depositar(150.0);

    std::cout << "Saldo actual: $" << miCuenta.getSaldo() << "\n";

    return 0;
}

```

5. ¿Qué es el enlace dinámico (late binding)?

El enlace dinámico (también conocido como late binding o vinculación tardía) es el mecanismo por el cual un programa asocia la llamada a un método con su definición exacta durante la ejecución (runtime) y no durante la compilación.

Es el pilar fundamental que permite el polimorfismo en la programación orientada a objetos.

¿Cómo funciona en C++?

Por defecto, C++ utiliza enlace estático (early binding) por eficiencia. Para activar el enlace dinámico, se deben cumplir tres condiciones:

- Usar herencia entre clases.
- Declarar los métodos con la palabra clave virtual en la clase base.
- Llamar a los métodos a través de punteros o referencias de la clase base.

Ejemplo práctico en C++

```
#include <iostream>

class Animal {
public:
    // La palabra 'virtual' activa el enlace dinámico
    virtual void hacerSonido() {
        std::cout << "El animal hace un sonido genérico.\n";
    }
};

class Perro : public Animal {
public:
    void hacerSonido() override {
        std::cout << "El perro ladra: ¡Guau!\n";
    }
};

class Gato : public Animal {
public:
    void hacerSonido() override {
        std::cout << "El gato maúlla: ¡Miau!\n";
    }
};

int main() {
    // Un puntero de tipo Animal puede apuntar a cualquier objeto
    derivado
    Animal* miAnimal;

    Perro miPerro;
    Gato miGato;

    // El programa decide EN TIEMPO DE EJECUCIÓN qué método llamar:
    miAnimal = &miPerro;
    miAnimal->hacerSonido(); // Imprime: ¡Guau!
```

```
miAnimal = &miGato;
miAnimal->hacerSonido(); // Imprime: ¡Miau!

return 0;
}
```

6. Diferencie entre sobrecarga y sobreescritura de métodos.

La diferencia principal entre sobrecarga y sobreescritura es el momento en que se resuelve la unión del método y el lugar donde se declaran.

Sobrecarga de métodos (Overloading)

Ocurre dentro de la misma clase. Consiste en definir varios métodos con el mismo nombre pero con diferente número o tipo de argumentos (diferente firma). El compilador decide cuál usar en tiempo de compilación.

- Ejemplo: calcular(int a) y calcular(double a, double b).

Sobreescritura de métodos (Overriding)

Ocurre en clases con relación de herencia. Consiste en redefinir en la clase hija un método que ya existe en la clase base, manteniendo exactamente la misma firma. El programa decide cuál usar en tiempo de ejecución (enlace dinámico).

- Ejemplo: La clase base Animal tiene hacerSonido() y la clase hija Perro redefine hacerSonido().

EJEMPLO

```
#include <iostream>

class Calculadora {
public:
    // SOBRECARGA: Mismo nombre, diferentes parámetros
    int sumar(int a, int b) {
        return a + b;
    }
    double sumar(double a, double b) {
        return a + b;
    }
};

class Padre {
public:
    virtual void presentarse() {
        std::cout << "Soy el Padre\n";
    }
};

class Hijo : public Padre {
public:
    // SOBREESCRITURA: Mismo nombre y misma firma en clase heredada
    void presentarse() override {
        std::cout << "Soy el Hijo\n";
    }
};
```

7. ¿Qué es una interfaz en términos conceptuales dentro de C++?

En términos conceptuales, una interfaz en C++ es un contrato que define qué puede hacer un objeto, pero no cómo lo hace.

Dado que C++ no tiene la palabra clave interface (como Java o C#), las interfaces se implementan mediante clases abstractas puras.

Características clave

- **Sin implementación:** No contiene código ni lógica en sus métodos.
- **Sin variables de datos:** Solo define declaraciones de funciones (comportamientos).
- **No instanciable:** No puedes crear un objeto directamente de ella (ej. Interfaz miObjeto; dará error).
- **Obligación:** Cualquier clase que herede de ella está obligada a programar el cuerpo de todos sus métodos.

Cómo se define en código C++

Para que una clase actúe estrictamente como una interfaz, debe cumplir dos reglas:

- Tener al menos un método virtual puro (igualado a 0).
- Tener un destructor virtual.

```
// Esto es conceptualmente una Interfaz
class Conectable {
public:
    // Método virtual puro (así se define en C++)
    virtual void conectar() = 0;
    virtual void desconectar() = 0;

    // Destructor virtual obligatorio para liberar memoria
    // correctamente
    virtual ~Conectable() {}
};
```

Ejemplo de uso práctico

Cualquier clase que herede de Conectable debe cumplir con el contrato:

```
class Router : public Conectable {
public:
    void conectar() override {
        // Lógica específica para un router
    }
    void desconectar() override {
        // Lógica específica para un router
    }
};

class BaseDeDatos : public Conectable {
public:
    void conectar() override {
        // Lógica específica para una base de datos
    }
    void desconectar() override {
        // Lógica específica para una base de datos
    }
};
```

8. Explique el uso de la palabra clave virtual.

La palabra clave virtual en C++ le indica al compilador que un método debe resolverse mediante enlace dinámico (en tiempo de ejecución) y no mediante enlace estático (en tiempo de compilación).

Es la herramienta técnica que habilita el polimorfismo y la creación de interfaces.

LOS 3 USOS PRINCIPALES DE VIRTUAL.

Métodos Virtuales Estándar (Polimorfismo)

Permite que una clase hija sobrescriba el comportamiento de un método de la clase base. Si un puntero de la clase base apunta a un objeto de la clase hija, el programa llamará al método del hijo.

- Sin virtual: El programa llama al método del padre (según el tipo de puntero).
- Con virtual: El programa busca el método del objeto real en la memoria.

Métodos Virtuales Puros (Interfaces)

Se declara igualando el método a cero (= 0). Indica que el método no tiene implementación en esa clase y obliga a las clases hijas a programarlo. Esto transforma a la clase en una clase abstracta o interfaz.

```
virtual void ejecutar() = 0; // Obligatorio implementar en clases hijas
```

Destrucciónes Virtuales (Gestión de Memoria)

Es obligatorio si una clase tiene métodos virtuales. Asegura que al destruir un objeto de la clase hija a través de un puntero de la clase base, se ejecute primero el destructor del hijo y luego el del padre, evitando fugas de memoria (memory leaks).

EJEMPLO: CON VIRTUAL Y SIN VIRTUAL

```
#include <iostream>

class Base {
public:
    void saludoNormal() { std::cout << "Base: Hola\n"; }
    virtual void saludoVirtual() { std::cout << "Base: Hola Virtual\n"; }

    virtual ~Base() {} // Destructor virtual para seguridad
};

class Derivada : public Base {
public:
    void saludoNormal() { std::cout << "Derivada: Qué tal\n"; }
    void saludoVirtual() override { std::cout << "Derivada: Qué tal Virtual\n"; }
};

int main() {
    Base* objeto = new Derivada();

    // CASO 1: Sin virtual (Enlace estático)
    objeto->saludoNormal(); // Imprime: "Base: Hola" (;Ignora al hijo!)
```

```
// CASO 2: Con virtual (Enlace dinámico)
objeto->saludoVirtual(); // Imprime: "Derivada: Qué tal Virtual"

delete objeto;
return 0;
}
```

9. ¿Qué es el principio de sustitución de Liskov?

El Principio de Sustitución de Liskov (LSP) es la tercera regla del acrónimo SOLID de diseño de software. Su definición formal establece que un objeto de una clase hija debe poder usarse en lugar de un objeto de su clase base sin que el programa falle o se comporte de forma inesperada.

En términos sencillos: si una función recibe como parámetro una clase padre, debes poder pasarle cualquier clase hija y la función debe seguir funcionando correctamente, sin necesidad de saber qué hijo exacto recibió.

10. ¿Qué problema resuelve el uso de punteros a clases base?

El uso de punteros a clases base resuelve el problema del acoplamiento rígido y la rigidez del código, permitiendo gestionar múltiples objetos de distintas clases derivadas bajo una interfaz única y genérica.

Sin esta capacidad, tendrías que escribir código duplicado para cada nuevo tipo de objeto que agregues a tu sistema.

Los 3 problemas específicos que resuelve

Duplicación de código (Colecciones heterogéneas)

Si tienes una lista de Perro, Gato y Pájaro, sin punteros a la clase base necesitarías un arreglo (array) diferente para cada tipo de animal. Con punteros a la clase base, puedes almacenar todos en un solo arreglo de tipo Animal*.

Código dependiente del futuro (Inflexibilidad)

Permite escribir funciones que funcionan con clases que aún no se han programado. Si tu función recibe un Animal*, seguirá funcionando dentro de tres meses cuando crees la clase Elefante, sin cambiar una sola línea de esa función.

El problema del "Slicing" (Recorte de objetos)

Si pasas un objeto derivado a una función por valor (copia) usando la clase base, C++ "recorta" y destruye toda la información y métodos del hijo. Los punteros (y las referencias) evitan esto, manteniendo la identidad real del objeto en memoria.

SECCIÓN II: ANÁLISIS DE CÓDIGO

II. Analice el siguiente código y explique la salida:

```
#include <iostream>
using namespace std;

class Base {
public :
    virtual void mostrar () {
        cout << "Base" << endl;
    }
};

class Derivada : public Base {
public :
    void mostrar () override {
        cout << "Derivada" << endl;
    }
};

int main () {
    Base* obj = new Derivada();
    obj->mostrar();
    return 0;
}
```

Explicación detallada

El resultado se debe a la combinación de tres factores clave de la Programación Orientada a Objetos en C++:

Uso de punteros de la clase base: En la línea `Base* obj = new Derivada();`, se crea un puntero de tipo `Base` que apunta a un objeto real de tipo `Derivada` en la memoria dinámica.

Método Virtual (virtual): En la clase `Base`, el método `mostrar()` está declarado con la palabra clave `virtual`. Esto le indica al compilador que no debe usar el enlace estático, sino activar el enlace dinámico (late binding).

Sobreescritura (override): La clase `Derivada` redefine el comportamiento del método mediante `override`. Al ejecutarse la línea `obj->mostrar();`, el programa busca en la tabla de funciones virtuales (VTable) del objeto real y ejecuta el método de la clase hija (`Derivada`) en lugar del de la clase padre (`Base`).

12. ¿Qué sucede si se elimina la palabra clave virtual en el método `mostrar()`?

Si se elimina la palabra clave `virtual` en el método `mostrar()` de la clase `Base`, la salida del programa cambia a `Base`.

Explicación

Enlace Estático (Early Binding): Al no existir la palabra virtual, C++ resuelve la llamada al método durante la compilación basándose estrictamente en el tipo del puntero (Base*), ignorando por completo el objeto real que está en memoria (Derivada).

Error de compilación adicional: El compilador arrojará un error en la clase Derivada debido a la palabra clave override. Como el método del padre ya no es virtual, la clase hija ya no puede "sobrescribirlo" legalmente bajo esa directiva.

13. Identifique y explique el error del siguiente código:

```
class A {  
public :  
    void metodo () = 0;  
};
```

El código presenta dos errores principales, uno de sintaxis y uno de concepto:

Error de Sintaxis (El principal): Intentar declarar un método virtual puro sin usar la palabra clave virtual. La sintaxis = 0 es exclusiva de las funciones virtuales.

Error Conceptual: Si la intención es crear una clase abstracta o interfaz, requiere obligatoriamente virtual para que las clases derivadas puedan implementar dicho método.

CODIGO CORREGIDO

```
class A {  
public:  
    // CORRECCIÓN: Se añade 'virtual' al inicio  
    virtual void metodo() = 0;  
};
```

14. Analice el siguiente código y determine si existe polimorfismo. Justifique su respuesta.

```
#include <iostream>  
using namespace std;  
  
class Animal {  
public :  
    void sonido () {  
        cout << "Animal hace sonido" << endl;  
    }  
};  
  
class Perro : public Animal {  
public :  
    void sonido () {  
        cout << "Perro ladra" << endl;  
    }  
};
```

No existe polimorfismo en el código presentado.

Justificación

Para que exista polimorfismo dinámico en C++ se deben cumplir tres condiciones obligatorias, y en este código falta la más importante:

Falta la palabra clave virtual: El método sonido() en la clase base Animal no está declarado como virtual. Por lo tanto, C++ aplica enlace estático (early binding) en lugar de enlace dinámico.

Consecuencia: Si crearas un puntero de la clase base apuntando al hijo (ej. Animal* an = new Perro();) y llamaras a an->sonido();, se ejecutaría el método de Animal ("Animal hace sonido") e ignoraría por completo al Perro.

Lo que ocurre en su lugar: Lo que hace la clase Perro no es polimorfismo, sino ocultamiento de métodos (method hiding o shadowing). El método del hijo solo se llamaría si usas directamente un objeto o puntero de tipo Perro.

15. ¿Cuál es la salida del siguiente programa?

```
#include <iostream>
using namespace std;

class A {
public:
    A () { cout << "A"; }
};

class B : public A {
public:
    B () { cout << "B"; }
};

int main () {
    B obj ;
}
```

La salida del programa es AB.

Explicación

El resultado se debe al orden en que se ejecutan los constructores en un esquema de herencia:

Creación del objeto: En la función main, la instrucción B obj; solicita la creación de un objeto de la clase hija (B).

Llamada en cascada: Antes de ejecutar el cuerpo del constructor de la clase hija (B), C++ está obligado a construir la base o "cimientos" del objeto. Por lo tanto, llama automáticamente primero al constructor de la clase base (A).

Ejecución: Se ejecuta el constructor de A e imprime A. Se termina de ejecutar el constructor de B e imprime B.

16. Explique el posible problema de memoria en la siguiente instrucción:

```
Base * obj = new Derivada ();
```

El posible problema de memoria en esa instrucción es una fuga de memoria (memory leak) parcial al momento de liberar el objeto.

Explicación del problema

Si intentas liberar la memoria utilizando la instrucción `delete obj;`, ocurrirá lo siguiente:

Falta el destructor virtual: Si la clase Base no tiene su destructor declarado como virtual, C++ aplicará enlace estático.

Destrucción incompleta: Al hacer el `delete`, solo se ejecutará el destructor de la clase Base y se ignorará por completo el destructor de la clase Derivada.

Consecuencia: Todos los recursos, variables dinámicas o memoria que la clase Derivada haya reservado internamente se quedarán atrapados en la memoria RAM del sistema hasta que el programa se cierre.

17. Analice si el siguiente código cumple con el principio de encapsulamiento:

```
class Cuenta {  
public :  
    double saldo ;  
};
```

No cumple con el principio de encapsulamiento.

Explicación del porqué

El encapsulamiento exige ocultar los detalles internos y el estado de un objeto para protegerlo de modificaciones externas no autorizadas o inválidas. Este código viola ese principio por lo siguiente:

Atributo Público (public): La variable `saldo` está declarada en la sección pública. Cualquier parte externa del programa puede acceder a ella y modificarla directamente de forma descontrolada (por ejemplo, haciendo `objeto.saldo = -50000;`).

Falta de control: No existen métodos de acceso seguro (Getters y Setters) que validen si los montos asignados son lógicos o correctos.

18. ¿Qué ocurre si un destructor no es virtual en un esquema de herencia?

Si un destructor no es virtual en un esquema de herencia, ocurre un comportamiento indefinido que generalmente provoca una fuga de memoria (memory leak)

19. ¿Qué función cumple la palabra clave override en C++?

La palabra clave override en C++ sirve para indicarle explícitamente al compilador que un método en una clase derivada tiene la intención de sobrescribir un método virtual de la clase base. Su función principal es actuar como una red de seguridad, obligando al compilador a generar un error si cometes un fallo en la firma del método (como escribir mal el nombre o cambiar un parámetro), evitando así que crees accidentalmente un método nuevo en lugar de sobrescribir el original

20. Analice la siguiente clase y determine qué tipo de clase es:

```
class Ejemplo {  
public :  
    virtual void accion () = 0;  
};
```

El código de la pregunta define una clase abstracta (también conocida conceptualmente en C++ como una interfaz).

Justificación

La clase es abstracta porque contiene el método virtual void accion() = 0;.

La expresión = 0 convierte a la función en un método virtual puro, lo cual genera los siguientes efectos en el programa:

No se puede instanciar: Está prohibido crear objetos directamente de esta clase (por ejemplo, Ejemplo obj; daría un error de compilación).

Obligación para los hijos: Cualquier clase que herede de Ejemplo estará obligada a implementar el cuerpo de la función accion() para poder generar objetos propios.

SECCIÓN III: DESARROLLO DE CÓDIGO

21. Cree una clase base Vehiculo con un método virtual mover().

```
#include <iostream>

// Clase base solicitada
class Vehiculo {
public:
    // Método virtual
    virtual void mover() {
        std::cout << "El vehiculo se esta moviendo de forma
generica.\n";
    }

    // Destructor virtual obligatorio para herencia
    virtual ~Vehiculo() {}
};

// Clase derivada para demostrar el polimorfismo
class Auto : public Vehiculo {
public:
    void mover() override {
        std::cout << "El auto avanza sobre cuatro ruedas por la
carretera.\n";
    }
};

int main() {
    // Uso de puntero a clase base (Concepto de la pregunta 10)
    Vehiculo* miVehiculo = new Auto();

    // Llama al método del Auto gracias a 'virtual' (Concepto de la
pregunta 11)
    miVehiculo->mover();

    // Liberación segura de memoria gracias al destructor virtual
    delete miVehiculo;

    return 0;
}
```

22. Cree dos clases derivadas: Auto y Bicicleta, que redefinan el método mover().

```
#include <iostream>

// Clase base (Pregunta 21)
class Vehiculo {
public:
    virtual void mover() {
        std::cout << "El vehículo se mueve.\n";
    }
    virtual ~Vehiculo() {} // Destructor virtual obligatorio
};

// Clase derivada 1: Auto
class Auto : public Vehiculo {
public:
    void mover() override {
        std::cout << "El auto acelera y avanza por la carretera.\n";
    }
};

// Clase derivada 2: Bicicleta
class Bicicleta : public Vehiculo {
public:
    void mover() override {
        std::cout << "La bicicleta avanza con el pedaleo.\n";
    }
};

int main() {
    // Demostración práctica usando punteros a la clase base
    Vehiculo* v1 = new Auto();
    Vehiculo* v2 = new Bicicleta();

    // Se ejecuta el método específico de cada hijo gracias a
    'virtual' y 'override'
    v1->mover(); // Imprime el mensaje del Auto
    v2->mover(); // Imprime el mensaje de la Bicicleta

    // Liberación de memoria limpia
    delete v1;
    delete v2;

    return 0;
}
```

23. Implemente un ejemplo de polimorfismo usando punteros.

```
#include <iostream>

// Clase Base
class Vehiculo {
public:
    virtual void mover() {
        std::cout << "El vehículo se mueve.\n";
    }
    virtual ~Vehiculo() {} // Destructor virtual obligatorio
};

// Clases Derivadas
class Auto : public Vehiculo {
public:
    void mover() override {
        std::cout << "El auto acelera en la carretera.\n";
    }
};

class Bicicleta : public Vehiculo {
public:
    void mover() override {
        std::cout << "La bicicleta avanza pedaleando.\n";
    }
};

int main() {
    // POLIMORFISMO: Creamos un arreglo de punteros a la clase base 'Vehiculo'
    Vehiculo* listaVehiculos[2];

    // Asignamos los objetos de las clases hijas a los punteros de la clase padre
    listaVehiculos[0] = new Auto();
    listaVehiculos[1] = new Bicicleta();

    std::cout << "--- Ejecutando Polimorfismo ---\n";

    // Tratamos a todos los objetos por igual, pero cada uno actúa según su tipo real
    for (int i = 0; i < 2; i++) {
        listaVehiculos[i]->mover();
    }

    // Limpieza de memoria dinámica
    delete listaVehiculos[0];
    delete listaVehiculos[1];

    return 0;
}
```


24. Cree una clase abstracta Figura con el método virtual puro area().

```
#include <iostream>

// Clase Abstracta (Funciona como Interfaz)
class Figura {
public:
    // El '= 0' define el método virtual puro
    virtual double area() = 0;

    // Destructor virtual para asegurar herencia limpia
    virtual ~Figura() {}
};

// Clase derivada obligada a implementar el método area()
class Cuadrado : public Figura {
private:
    double lado;
public:
    Cuadrado(double l) : lado(l) {}

    // Implementación obligatoria del contrato de Figura
    double area() override {
        return lado * lado;
    }
};

int main() {
    // Intentar hacer: Figura f; daría un ERROR de compilación (es abstracta)

    // Creamos un puntero a la clase base apuntando al objeto derivado
    Figura* miFigura = new Cuadrado(5.0);

    std::cout << "El area del cuadrado es: " << miFigura->area() <<
std::endl; // Imprime 25

    delete miFigura;
    return 0;
}
```

25. Implemente las clases Rectangulo y Circulo heredando de Figura.

```
#include <iostream>

// Clase Abstracta (Pregunta 24)
class Figura {
public:
    virtual double area() = 0; // Método virtual puro
    virtual ~Figura() {}       // Destructor virtual
};

// Clase Derivada 1: Rectangulo
class Rectangulo : public Figura {
private:
    double base;
    double altura;
public:
    Rectangulo(double b, double a) : base(b), altura(a) {}

    double area() override {
        return base * altura;
    }
};

// Clase Derivada 2: Circulo
class Circulo : public Figura {
private:
    double radio;
public:
    Circulo(double r) : radio(r) {}

    double area() override {
        // Fórmula:  $\pi * r^2$  (Usamos 3.1416 para mantenerlo simple)
        return 3.1416 * radio * radio;
    }
};

int main() {
    // Instanciamos los objetos usando punteros de la clase base
    Figura
    Figura* fig1 = new Rectangulo(5.0, 4.0); // Base = 5, Altura = 4
    Figura* fig2 = new Circulo(3.0);         // Radio = 3

    std::cout << "Area del rectangulo: " << fig1->area() << "\n"; //
    Imprime 20
    std::cout << "Area del circulo: " << fig2->area() << "\n";    //
    Imprime 28.2744

    // Liberación de memoria dinámica
    delete fig1;
    delete fig2;

    return 0;
}
```

26. Cree un ejemplo de sobrecarga de funciones.

```
#include <iostream>
#include <string>

// 1. Versión para números enteros
void imprimir(int numero) {
    std::cout << "Imprimiendo un entero: " << numero << "\n";
}

// 2. Versión para números decimales
void imprimir(double decimal) {
    std::cout << "Imprimiendo un decimal: " << decimal << "\n";
}

// 3. Versión para texto corriente
void imprimir(std::string texto) {
    std::cout << "Imprimiendo un texto: " << texto << "\n";
}

int main() {
    // El compilador decide qué función usar en tiempo de compilación
    según el argumento
    imprimir(10);           // Llama a la versión 1
    imprimir(5.75);         // Llama a la versión 2
    imprimir("Hola C++");   // Llama a la versión 3

    return 0;
}
```

27. Cree una clase con atributos privados y métodos públicos, aplicando correctamente el encapsulamiento.

```
#include <iostream>
#include <string>

class Producto {
private:
    // Atributos PRIVADOS: Nadie fuera de esta clase puede
    modificarlos directamente
    std::string nombre;
    double precio;
public:
    // Constructor para inicializar de forma segura
    Producto(std::string nombreInicial, double precioInicial) {
        nombre = nombreInicial;
        setPrecio(precioInicial); // Usamos el validador desde el
        inicio
    }

    // MÉTODO PÚBLICO (Getter): Permite leer el precio de forma segura
    double getPrecio() const {
        return precio;
    }

    // MÉTODO PÚBLICO (Setter): Permite modificar el precio BAJO
    REGLAS
    void setPrecio(double nuevoPrecio) {
        if (nuevoPrecio >= 0) {
            precio = nuevoPrecio; // Solo se asigna si el valor es
            lógico
        } else {
            std::cout << "Error: El precio no puede ser negativo. Se
            asignara $0.\n";
            precio = 0;
        }
    }
};

int main() {
    // Creación correcta del objeto
    Producto miArticulo("Laptop", 800.0);

    // INTENTO INCORRECTO: miArticulo.precio = -150;
    // Error de compilación: 'precio' es privado. El código está
    protegido.

    // FORMA CORRECTA: Intentamos asignar un precio inválido mediante
    el método público
    miArticulo.setPrecio(-50.0);

    std::cout << "Precio final del producto: $" <<
    miArticulo.getPrecio() << "\n";

    return 0;
}
```

28. Implemente un constructor y un destructor que muestren mensajes en pantalla.

```
#include <iostream>

class Componente {
public:
    // Constructor: Se ejecuta automáticamente al CREAR el objeto
    Componente() {
        std::cout << "--> Constructor: Objeto creado y recursos
asignados.\n";
    }

    // Destructor: Se ejecuta automáticamente al DESTRUIR el objeto
    ~Componente() {
        std::cout << "<-- Destructor: Objeto destruido y memoria
liberada.\n";
    }
};

int main() {
    std::cout << "[Inicio del main]\n";

    {
        // Creamos un objeto dentro de un bloque aislado (ámbito
local)
        Componente miObjeto;
        std::cout << "...El objeto esta vivo y siendo utilizado...\n";
    } // Al llegar aquí, el objeto sale de su ámbito y se destruye
automáticamente

    std::cout << "[Fin del main]\n";
    return 0;
}
```

29. Cree un ejemplo donde se utilice late binding.

```
#include <iostream>

class MensajeBase {
public:
    // La palabra clave 'virtual' activa el late binding
    virtual void mostrar() {
        std::cout << "Mensaje desde la clase Base.\n";
    }
    virtual ~MensajeBase() {} // Destructor virtual por seguridad
};

class MensajeDerivado : public MensajeBase {
public:
    // Sobreescritura del método
    void mostrar() override {
        std::cout << "Mensaje desde la clase Derivada (Late Binding
funcionando).\n";
    }
};

int main() {
    // 1. Creamos un puntero de tipo clase Base
    MensajeBase* puntero;

    // 2. Lo hacemos apuntar a un objeto de la clase Derivada
    puntero = new MensajeDerivado();

    // LATE BINDING: El programa no sabe qué método llamar en la
    compilación.
    // Revisa el objeto real en la memoria DURANTE LA EJECUCIÓN y
    llama al hijo.
    puntero->mostrar();

    // Limpieza de memoria
    delete puntero;
    return 0;
}
```

30. Desarrolle un mini sistema con las siguientes características:

- Clase base: Empleado
- Clases derivadas: Gerente, Obrero
- Método virtual: calcularSalario()

```
#include <iostream>
#include <string>
#include <vector>

// 1. Clase Base
class Empleado {
protected: // Permite que las clases hijas accedan directamente a
    estos atributos
    std::string nombre;
```

```

    double salarioBase;

public:
    Empleado(std::string nom, double salario) : nombre(nom),
    salarioBase(salario) {}

    // Método virtual para activar el enlace dinámico (Late Binding)
    virtual double calcularSalario() {
        return salarioBase;
    }

    // Getter para el nombre
    std::string getNombre() const { return nombre; }

    // Destructor virtual obligatorio para liberar memoria
    correctamente
    virtual ~Empleado() {}
};

// 2. Clase Derivada: Gerente (Recibe un bono fijo adicional)
class Gerente : public Empleado {
private:
    double bono;

public:
    Gerente(std::string nom, double salario, double b) : Empleado(nom,
    salario), bono(b) {}

    // Sobreescritura del método
    double calcularSalario() override {
        return salarioBase + bono;
    }
};

// 3. Clase Derivada: Obrero (Recibe un pago extra por horas
// trabajadas)
class Obrero : public Empleado {
private:
    int horasExtras;
    double pagoPorHora;

public:
    Obrero(std::string nom, double salario, int horas, double pago)
        : Empleado(nom, salario), horasExtras(horas),
    pagoPorHora(pago) {}

    // Sobreescritura del método
    double calcularSalario() override {
        return salarioBase + (horasExtras * pagoPorHora);
    }
};

// 4. Función Principal (Demostración de Polimorfismo)
int main() {
    // Creamos una colección heterogénea usando punteros a la clase
    base
    std::vector<Empleado*> nomina;

```

```

    // Agregamos un Gerente y un Obrero al sistema
    nomina.push_back(new Gerente("Carlos Alvarado", 2500.0, 600.0));
// Base 2500 + Bono 600
    nomina.push_back(new Obrero("Juan Gomez", 1200.0, 10, 15.0));
// Base 1200 + (10 * 15)

    std::cout << "=== REPORTE DE NOMINA INTERNA ===\n\n";

    // Procesamos la nómina de forma genérica usando polimorfismo
    for (Empleado* emp : nomina) {
        std::cout << "Empleado: " << emp->getNombre() << "\n";
        std::cout << "Salario Total: $" << emp->calcularSalario() <<
"\n";
        std::cout << "-----\n";
    }

    // Limpieza de memoria dinámica para evitar fugas (Memory Leaks)
    for (Empleado* emp : nomina) {
        delete emp;
    }

    return 0;
}

```

Bibliografía

C++ virtual functions. (s/f). W3schools.com. Recuperado el 25 de mayo de 2026, de https://www.w3schools.com/cpp/cpp_virtual_functions.asp

C# best practices - dangers of violating SOLID principles in C#. (s/f). Microsoft.com. Recuperado el 25 de mayo de 2026, de <https://learn.microsoft.com/en-us/archive/msdn-magazine/2014/may/csharp-best-practices-dangers-of-violating-solid-principles-in-csharp>

Cutting edge - code contracts: Inheritance and the liskov principle. (s/f). Microsoft.com. Recuperado el 25 de mayo de 2026, de <https://learn.microsoft.com/en-us/archive/msdn-magazine/2011/july/msdn-magazine-cutting-edge-code-contracts-inheritance-and-the-liskov-principle>

Llamas, L. (2024, noviembre 19). *Qué es y cómo usar polimorfismo en C++*. Luis Llamas. <https://www.luisllamas.es/cpp-polimorfismo/>

Polimorfismo en C++. (s/f). Blogspot.com. Recuperado el 25 de mayo de 2026, de <https://cucarachasracing.blogspot.com/2014/02/polimorfismo-en-c.html>

Polymorphism in C++ and types of polymorphism. (2021, febrero 1). Great Learning Blog: Free Resources What Matters to Shape Your Career!;

Great Learning. <https://www.mygreatlearning.com/blog/polymorphism-in-cpp/>

(S/f). Cppreference.com. Recuperado el 25 de mayo de 2026, de <https://en.cppreference.com/cpp/language/virtual>

Botella, R. D. (s/f). *UN ENFOQUE PRÁCTICO*. Elhacker.info. Recuperado el 25 de mayo de 2026, de <https://elhacker.info/manuales/Lenguajes%20de%20Programacion/C++/Programacion-Orientada-a-Objetos-en-Cplusplus.pdf>

CICS Transaction Server for z/OS. (2026, febrero 19). Ibm.com. <https://www.ibm.com/docs/es/cics-ts/6.x?topic=classes-polymorphic-behavior>

de clase., N. (s/f). *PROGRAMACIÓN ORIENTADA A OBJETOS CON C++*. Utm.mx. Recuperado el 25 de mayo de 2026, de [https://www.utm.mx/~caff/doc/Notas%20POO%20\(2000\).pdf](https://www.utm.mx/~caff/doc/Notas%20POO%20(2000).pdf)

Investigación, L., & Servicios, C. P. D. (s/f). *Manual básico de Programación en C++*. Wikimedia.org. Recuperado el 25 de mayo de 2026, de https://upload.wikimedia.org/wikipedia/commons/a/ab/Manual_C%2B%2B.PDF

Velazquez, J. L. A. (s/f). *Lenguaje de Programación: C++ POO (Programación Orientada a Objetos)*. Cimat.mx. Recuperado el 25 de mayo de 2026, de https://www.cimat.mx/~pepe/cursos/lenguaje_2010/slides/slide_40.pdf